REPUBLIQUE TUNISIENNE
* * *
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE
* * *
UNIVERSITE DE SFAX
* * *
FACULTE DES SCIENCES DE SFAX

Compte rendu

Travaux Dirigés & Rendu

Problème de transport (recherche taboue)
Problème du voyageur de commerce (recherche taboue, recuit simulé, algorithme génétique)

Étudiante : Bouassida Chayma
Filière : Génie logiciel et systèmes d'information – 3e année
Année universitaire : 2026/2027

Exercice 2 : problème de transport

1. Énoncé et hypothèses

Deux dépôts D1 et D2 livrent trois magasins M1, M2, M3. La colonne « Dépôt » du tableau est interprétée comme le **stock** de chaque dépôt (8 et 9 containers). Les demandes sont 4, 5 et 8 containers. L'offre totale (8 + 9 = 17) égale la demande totale (4 + 5 + 8 = 17) : le problème est **équilibré**.

	M1	M2	M3	Stock
D1	5	3	4	8
D2	6	7	2	9
Demande	4	5	8	17

2. Modélisation en programmation linéaire en nombres entiers

Soit x_ij le nombre de containers envoyés du dépôt Di au magasin Mj et c_ij le coût unitaire de transport.

```
min  Z = 5 x11 + 3 x12 + 4 x13 + 6 x21 + 7 x22 + 2 x23

s.c. x11 + x12 + x13 = 8        (stock de D1)
     x21 + x22 + x23 = 9        (stock de D2)
     x11 + x21 = 4              (demande de M1)
     x12 + x22 = 5              (demande de M2)
     x13 + x23 = 8              (demande de M3)
     x_ij entier >= 0
```

Branch and Bound. La matrice des contraintes d'un problème de transport est totalement unimodulaire : la relaxation linéaire possède une solution optimale entière. Le Branch and Bound s'arrête donc à la racine, sans branchement. Ici, l'énumération complète de toutes les solutions réalisables donne le même optimum que celui trouvé par la recherche taboue (Z = 52).

3. Recherche taboue

Élément	Choix
Représentation	matrice x (2 x 3) d'entiers respectant stocks et demandes
Solution initiale	règle du coin nord-ouest, coût 55
Voisinage	cycle 2x2 : +1 en (i1,j1) et (i2,j2), -1 en (i1,j2) et (i2,j1) ; reste réalisable
Liste taboue	taille 3, interdit le mouvement inverse
Aspiration	un mouvement tabou est autorisé s'il améliore le meilleur coût
Arrêt	10 itérations

Capture d'écran de l'exécution :

```
Solution initiale (coin nord-ouest) : [[4, 4, 0], [0, 1, 8]]  cout = 55
Iter  1 | mouvement (0, 1, 1, 0) | cout = 52 | meilleur = 52
Iter  2 | mouvement (0, 1, 2, 0) | cout = 55 | meilleur = 52
Iter  3 | mouvement (1, 0, 2, 0) | cout = 52 | meilleur = 52
Iter  4 | mouvement (0, 1, 2, 0) | cout = 55 | meilleur = 52
Iter  5 | mouvement (1, 0, 2, 0) | cout = 52 | meilleur = 52
Iter  6 | mouvement (0, 1, 0, 1) | cout = 55 | meilleur = 52
Iter  7 | mouvement (1, 0, 0, 1) | cout = 52 | meilleur = 52
Iter  8 | mouvement (0, 1, 0, 1) | cout = 55 | meilleur = 52
Iter  9 | mouvement (1, 0, 0, 1) | cout = 52 | meilleur = 52
Iter 10 | mouvement (0, 1, 0, 1) | cout = 55 | meilleur = 52

Meilleure solution :
  D1 -> M1,M2,M3 = [3, 5, 0]
  D2 -> M1,M2,M3 = [1, 0, 8]
Cout minimal trouve : 52
```

Figure 1 - Exécution de exercice2.py (VS Code)

Résultat. Le coût passe de 55 à **52** dès la première itération. Ensuite la recherche oscille entre 52 et 55 : l'espace des solutions est très petit, donc quelques itérations suffisent.

	M1	M2	M3
D1	3	5	0
D2	1	0	8
Coût total			52

Exercice 3 : voyageur de commerce

1. Données

Le graphe compte 8 villes ; un tiret dans le tableau de l'énoncé signifie qu'il n'y a pas d'arête. Les distances sont supposées symétriques. Une arête inexistante reçoit une pénalité de 1000, ce qui rend le tour non réalisable. La solution initiale 1-2-3-4-5-6-7-8-1 est réalisable, de longueur 14 + 13 + 11 + 11 + 15 + 9 + 13 + 17 = **103**.

	1	2	3	4	5	6	7	8
1	-	14	15	x	x	x	x	17
2	14	-	13	14	20	x	x	21
3	15	13	-	11	21	17	9	9
4	x	14	11	-	11	10	8	20
5	x	20	21	11	-	15	18	x
6	x	x	17	10	15	-	9	x
7	x	x	9	8	18	9	-	13
8	17	21	9	20	x	x	13	-

(x = pas d'arête ; - = diagonale)

2. Recherche taboue

Élément	Choix
Solution initiale	1-2-3-4-5-6-7-8-1 (L = 103)
Voisinage	échange de deux villes (ville 1 fixe), 21 voisins
Liste taboue	taille 4, paires de villes échangées
Aspiration	autorisée si la longueur bat le meilleur tour
Arrêt	15 itérations

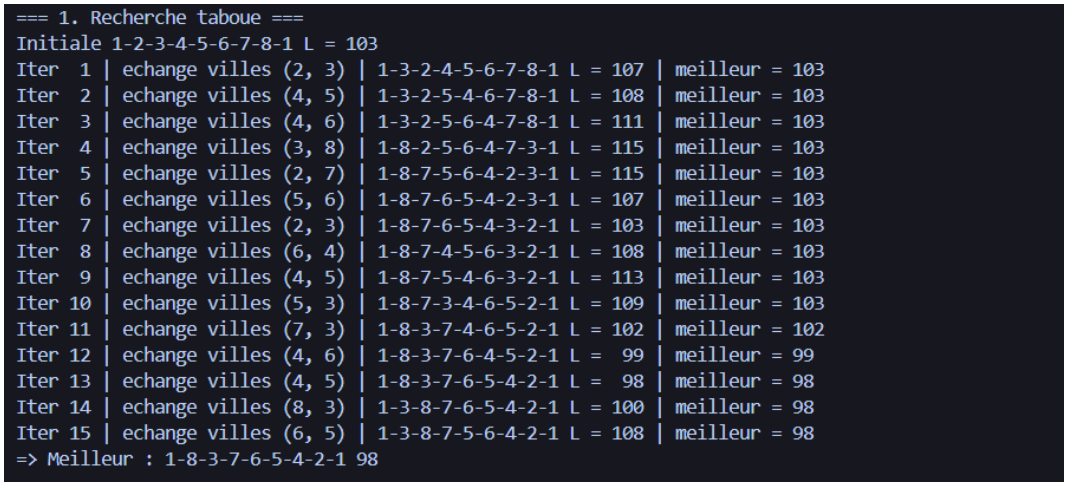


Figure 2 - Exécution de exercice3.py (VS Code) : recherche taboue

Résultat : 1-8-3-7-6-5-4-2-1, longueur **98**. La recherche accepte d'abord des tours plus longs (107, 108, 111, 115) grâce à la liste taboue, puis trouve 102, 99 et 98 aux itérations 11 à 13.

3. Recuit simulé

Paramètres : T0 = 20, décroissance géométrique T devient 0,5 T (variation empirique), 5 itérations, graine aléatoire 1. Voisin : échange de deux villes, en rejetant les échanges qui créent une arête inexistante. Règle de Metropolis : un tour plus long est accepté si r < exp(-delta/T), avec r tiré uniformément dans [0,1].

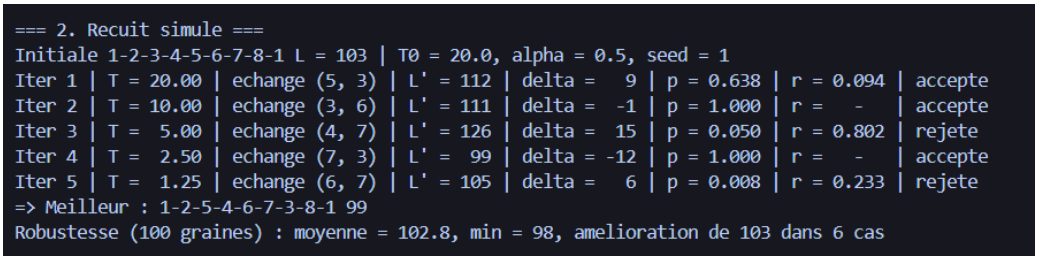


Figure 3 - Exécution de exercice3.py (VS Code) : recuit simulé

Résultat : 1-2-5-4-6-7-3-8-1, longueur **99**. À l'itération 1 (T = 20), un tour plus long (+9) est accepté avec une probabilité de 0,64, ce qui permet de s'échapper de la solution initiale. Aux itérations 3 et 5, T est plus faible et les dégradations sont rejetées.

Limite importante. Avec seulement 5 itérations, le résultat dépend fortement du hasard. Sur 100 graines différentes, la longueur moyenne obtenue est 102,8 et seulement 6 exécutions améliorent 103 (meilleur cas : 98). Le résultat 99 ci-dessus est donc un bon cas, pas un résultat typique. Avec plus d'itérations, le recuit serait bien plus fiable.

4. Algorithme génétique

Élément	Choix
Codage	permutation des villes, ville 1 en tête
Population	10 individus aléatoires, 30 générations
Sélection	la meilleure moitié (5 individus) survit
Croisement	Order Crossover (OX) entre deux parents de la moitié survivante
Mutation	échange de deux villes, probabilité 0,2
Fitness	longueur du tour (pénalité pour arête inexistante)

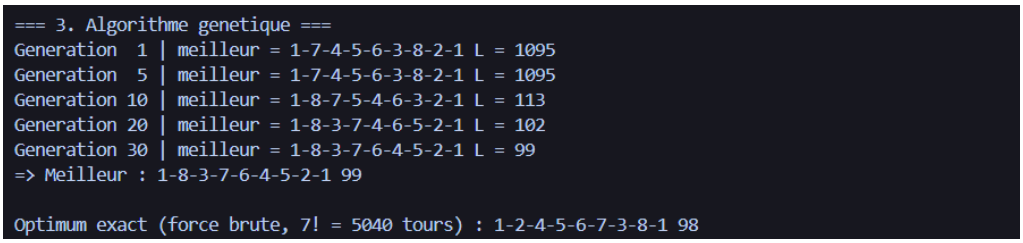


Figure 4 - Exécution de exercice3.py (VS Code) : algorithme génétique et optimum exact

Résultat : 1-8-3-7-6-4-5-2-1, longueur **99**. Aux générations 1 et 5, le meilleur individu est encore infaisable (L = 1095, une arête inexistante) ; il devient réalisable vers la génération 10 (113), puis la population converge vers 102 et 99.

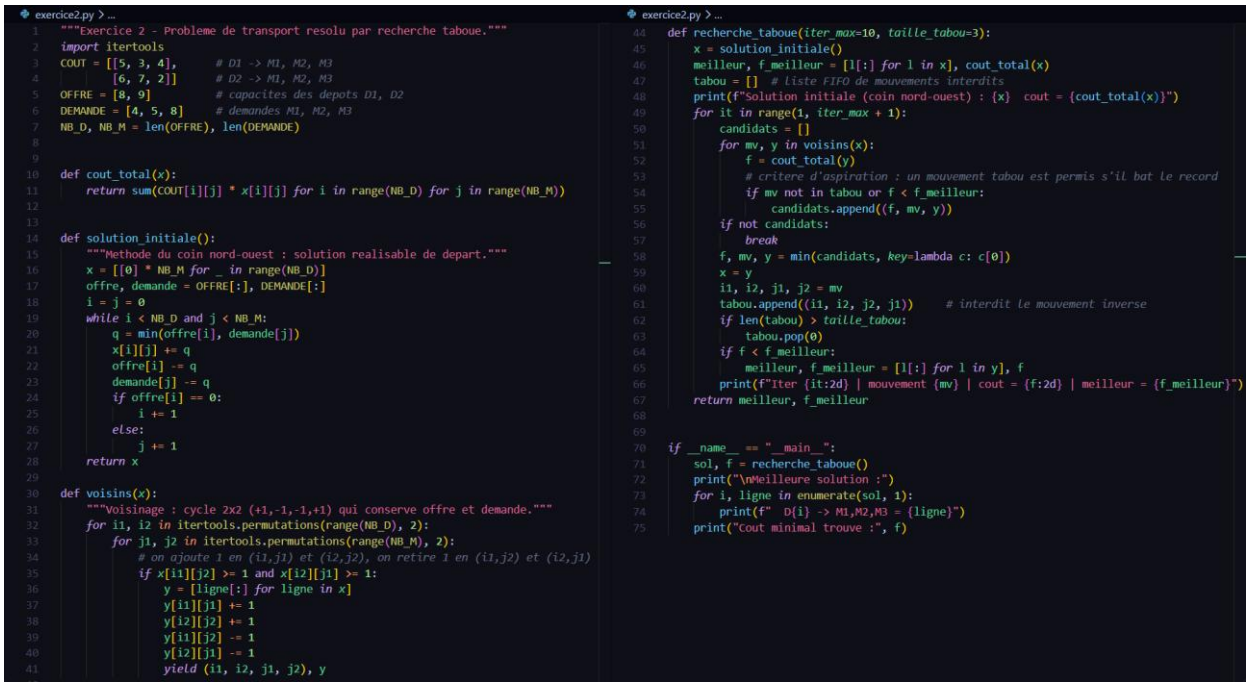
5. Comparaison

Méthode	Meilleur tour	Longueur	Remarque
Solution initiale	1-2-3-4-5-6-7-8-1	103	
Recherche taboue	1-8-3-7-6-5-4-2-1	98	optimum atteint, déterministe
Recuit simulé	1-2-5-4-6-7-3-8-1	99	dépend de la graine (5 itérations)
Algorithme génétique	1-8-3-7-6-4-5-2-1	99	à 1 de l'optimum
Force brute (5040 tours)	1-2-4-5-6-7-3-8-1	98	optimum exact

L'optimum exact, obtenu par énumération des 7! = 5040 tours, vaut 98. La recherche taboue l'atteint avec un autre tour de même longueur (1-8-3-7-6-5-4-2-1 : 17 + 9 + 9 + 9 + 15 + 11 + 14 + 14 = 98). Ces métaheuristiques ne garantissent pas l'optimum en général ; sur une instance de 8 villes la force brute reste possible, mais elle devient impraticable quand le nombre de villes augmente.

Annexe : captures d'écran du code

exercice2.py



```python
"""Exercice 2 - Probleme de transport resolu par recherche taboue."""
import itertools
COUT = [[5, 3, 4],      # D1 -> M1, M2, M3
        [6, 7, 2]]      # D2 -> M1, M2, M3
OFFRE = [8, 9]          # capacites des depots D1, D2
DEMANDE = [4, 5, 8]     # demandes M1, M2, M3
NB_D, NB_M = len(OFFRE), len(DEMANDE)


def cout_total(x):
    return sum(COUT[i][j] * x[i][j] for i in range(NB_D) for j in range(NB_M))


def solution_initiale():
    """Methode du coin nord-ouest : solution realisable de depart."""
    x = [[0] * NB_M for _ in range(NB_D)]
    offre, demande = OFFRE[:], DEMANDE[:]
    i = j = 0
    while i < NB_D and j < NB_M:
        q = min(offre[i], demande[j])
        x[i][j] += q
        offre[i] -= q
        demande[j] -= q
        if offre[i] == 0:
            i += 1
        else:
            j += 1
    return x

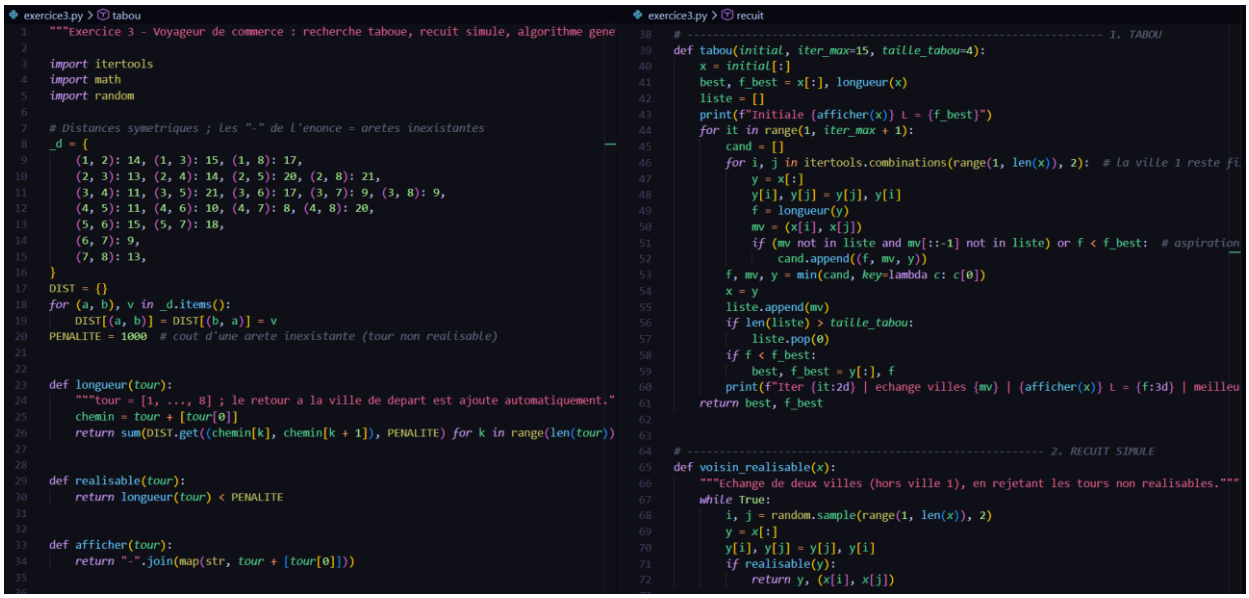
def voisins(x):
    """Voisinage : cycle 2x2 (+1,-1,-1,+1) qui conserve offre et demande."""
    for i1, i2 in itertools.permutations(range(NB_D), 2):
        for j1, j2 in itertools.permutations(range(NB_M), 2):
            # on ajoute 1 en (i1,j1) et (i2,j2), on retire 1 en (i1,j2) et (i2,j1)
            if x[i1][j2] >= 1 and x[i2][j1] >= 1:
                y = [ligne[:] for ligne in x]
                y[i1][j1] += 1
                y[i2][j2] += 1
                y[i1][j2] -= 1
                y[i2][j1] -= 1
                yield (i1, i2, j1, j2), y
```

```python
def recherche_taboue(iter_max=10, taille_tabou=3):
    x = solution_initiale()
    meilleur, f_meilleur = [l[:] for l in x], cout_total(x)
    tabou = []  # liste FIFO de mouvements interdits
    print(f"Solution initiale (coin nord-ouest) : {x}  cout = {cout_total(x)}")
    for it in range(1, iter_max + 1):
        candidats = []
        for mv, y in voisins(x):
            f = cout_total(y)
            # critere d'aspiration : un mouvement tabou est permis s'il bat le record
            if mv not in tabou or f < f_meilleur:
                candidats.append((f, mv, y))
        if not candidats:
            break
        f, mv, y = min(candidats, key=lambda c: c[0])
        x = y
        i1, i2, j1, j2 = mv
        tabou.append((i1, i2, j2, j1))     # interdit le mouvement inverse
        if len(tabou) > taille_tabou:
            tabou.pop(0)
        if f < f_meilleur:
            meilleur, f_meilleur = [l[:] for l in y], f
        print(f"Iter {it:2d} | mouvement {mv} | cout = {f:2d} | meilleur = {f_meilleur}")
    return meilleur, f_meilleur


if __name__ == "__main__":
    sol, f = recherche_taboue()
    print("\nMeilleure solution :")
    for i, ligne in enumerate(sol, 1):
        print(f"  D{i} -> M1,M2,M3 = {ligne}")
    print("Cout minimal trouve :", f)
```

Figure 5 - Code de exercice2.py

exercice3.py



```python
"""Exercice 3 - Voyageur de commerce : recherche taboue, recuit simule, algorithme gene

import itertools
import math
import random

# Distances symetriques ; les "-" de l'enonce = aretes inexistantes
_d = {
    (1, 2): 14, (1, 3): 15, (1, 8): 17,
    (2, 3): 13, (2, 4): 14, (2, 5): 20, (2, 8): 21,
    (3, 4): 11, (3, 5): 21, (3, 6): 17, (3, 7): 9, (3, 8): 9,
    (4, 5): 11, (4, 6): 10, (4, 7): 8, (4, 8): 20,
    (5, 6): 15, (5, 7): 18,
    (6, 7): 9,
    (7, 8): 13,
}
DIST = {}
for (a, b), v in _d.items():
    DIST[(a, b)] = DIST[(b, a)] = v
PENALITE = 1000  # cout d'une arete inexistante (tour non realisable)


def longueur(tour):
    """tour = [1, ..., 8] ; le retour a la ville de depart est ajoute automatiquement."
    chemin = tour + [tour[0]]
    return sum(DIST.get((chemin[k], chemin[k + 1]), PENALITE) for k in range(len(tour))

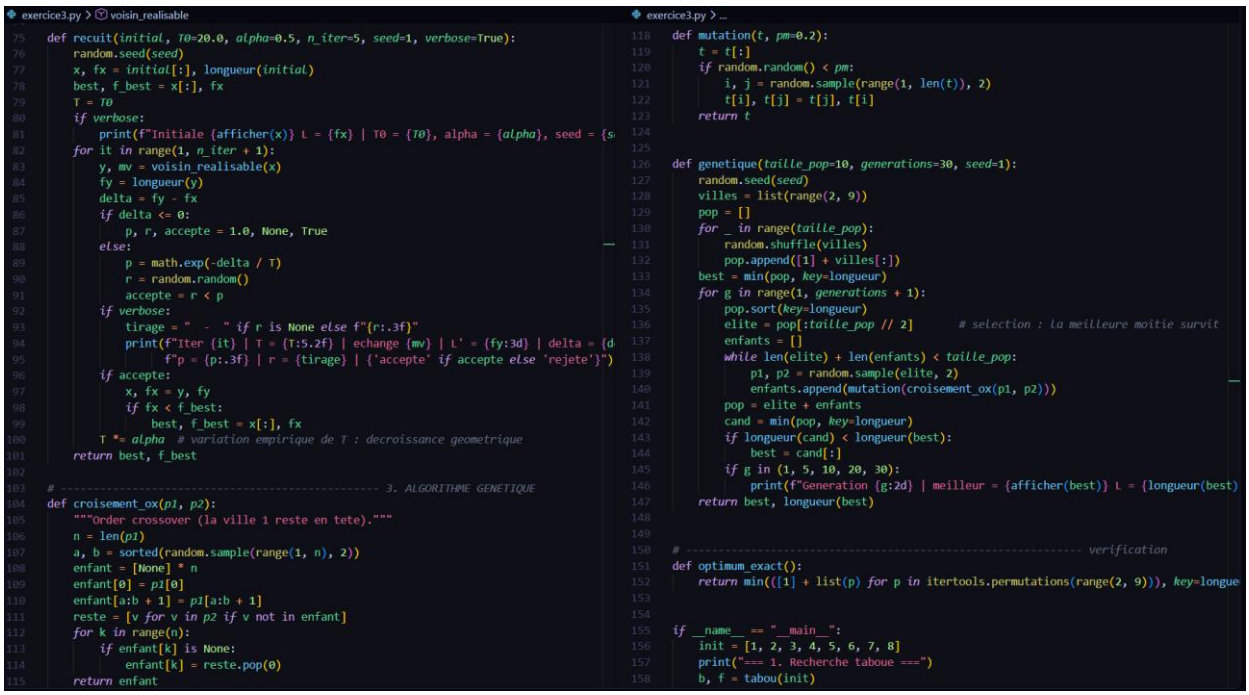

def realisable(tour):
    return longueur(tour) < PENALITE


def afficher(tour):
    return "-".join(map(str, tour + [tour[0]]))
```

```python
# ---------------------------------------------------------- 1. TABOU
def tabou(initial, iter_max=15, taille_tabou=4):
    x = initial[:]
    best, f_best = x[:], longueur(x)
    liste = []
    print(f"Initiale {afficher(x)} L = {f_best}")
    for it in range(1, iter_max + 1):
        cand = []
        for i, j in itertools.combinations(range(1, len(x)), 2):  # la ville 1 reste fi
            y = x[:]
            y[i], y[j] = y[j], y[i]
            f = longueur(y)
            mv = (x[i], x[j])
            if (mv not in liste and mv[::-1] not in liste) or f < f_best:  # aspiration
                cand.append((f, mv, y))
        f, mv, y = min(cand, key=lambda c: c[0])
        x = y
        liste.append(mv)
        if len(liste) > taille_tabou:
            liste.pop(0)
        if f < f_best:
            best, f_best = y[:], f
        print(f"Iter {it:2d} | echange villes {mv} | {afficher(x)} L = {f:3d} | meilleu
    return best, f_best


# ---------------------------------------------------------- 2. RECUIT SIMULE
def voisin_realisable(x):
    """Echange de deux villes (hors ville 1), en rejetant les tours non realisables."""
    while True:
        i, j = random.sample(range(1, len(x)), 2)
        y = x[:]
        y[i], y[j] = y[j], y[i]
        if realisable(y):
            return y, (x[i], x[j])
```

Figure 6 - exercice3.py : données, longueur d'un tour, recherche taboue (début du recuit simulé)



```python
def recuit(initial, T0=20.0, alpha=0.5, n_iter=5, seed=1, verbose=True):
    random.seed(seed)
    x, fx = initial[:], longueur(initial)
    best, f_best = x[:], fx
    T = T0
    if verbose:
        print(f"Initiale {afficher(x)} L = {fx} | T0 = {T0}, alpha = {alpha}, seed = {s
    for it in range(1, n_iter + 1):
        y, mv = voisin_realisable(x)
        fy = longueur(y)
        delta = fy - fx
        if delta <= 0:
            p, r, accepte = 1.0, None, True
        else:
            p = math.exp(-delta / T)
            r = random.random()
            accepte = r < p
        if verbose:
            tirage = "  -  " if r is None else f"{r:.3f}"
            print(f"Iter {it} | T = {T:5.2f} | echange {mv} | L' = {fy:3d} | delta = {d
                  f"p = {p:.3f} | r = {tirage} | {'accepte' if accepte else 'rejete'}")
        if accepte:
            x, fx = y, fy
            if fx < f_best:
                best, f_best = x[:], fx
        T *= alpha  # variation empirique de T : decroissance geometrique
    return best, f_best

# ---------------------------------------------- 3. ALGORITHME GENETIQUE
def croisement_ox(p1, p2):
    """Order crossover (la ville 1 reste en tete)."""
    n = len(p1)
    a, b = sorted(random.sample(range(1, n), 2))
    enfant = [None] * n
    enfant[0] = p1[0]
    enfant[a:b + 1] = p1[a:b + 1]
    reste = [v for v in p2 if v not in enfant]
    for k in range(n):
        if enfant[k] is None:
            enfant[k] = reste.pop(0)
    return enfant
```

```python
def mutation(t, pm=0.2):
    t = t[:]
    if random.random() < pm:
        i, j = random.sample(range(1, len(t)), 2)
        t[i], t[j] = t[j], t[i]
    return t


def genetique(taille_pop=10, generations=30, seed=1):
    random.seed(seed)
    villes = list(range(2, 9))
    pop = []
    for _ in range(taille_pop):
        random.shuffle(villes)
        pop.append([1] + villes[:])
    best = min(pop, key=longueur)
    for g in range(1, generations + 1):
        pop.sort(key=longueur)
        elite = pop[:taille_pop // 2]       # selection : la meilleure moitie survit
        enfants = []
        while len(elite) + len(enfants) < taille_pop:
            p1, p2 = random.sample(elite, 2)
            enfants.append(mutation(croisement_ox(p1, p2)))
        pop = elite + enfants
        cand = min(pop, key=longueur)
        if longueur(cand) < longueur(best):
            best = cand[:]
        if g in (1, 5, 10, 20, 30):
            print(f"Generation {g:2d} | meilleur = {afficher(best)} L = {longueur(best)
    return best, longueur(best)


# ------------------------------------------------------------ verification
def optimum_exact():
    return min(([1] + list(p) for p in itertools.permutations(range(2, 9))), key=longue


if __name__ == "__main__":
    init = [1, 2, 3, 4, 5, 6, 7, 8]
    print("=== 1. Recherche taboue ===")
    b, f = tabou(init)
```

Figure 7 - exercice3.py : recuit simulé, algorithme génétique

exercice3.py > voisin_realisable

```python
if __name__ == "__main__":
    init = [1, 2, 3, 4, 5, 6, 7, 8]
    print("=== 1. Recherche taboue ===")
    b, f = tabou(init)
    print("=> Meilleur :", afficher(b), f)

    print("\n=== 2. Recuit simule ===")
    b, f = recuit(init)
    print("=> Meilleur :", afficher(b), f)
    res = [recuit(init, seed=s, verbose=False)[1] for s in range(1, 101)]
    print(f"Robustesse (100 graines) : moyenne = {sum(res)/len(res):.1f}, "
          f"min = {min(res)}, amelioration de 103 dans {sum(r < 103 for r in res)} cas")

    print("\n=== 3. Algorithme genetique ===")
    b, f = genetique()
    print("=> Meilleur :", afficher(b), f)

    opt = optimum_exact()
    print("\nOptimum exact (force brute, 7! = 5040 tours) :", afficher(opt), longueur(opt))
```

Figure 8 - exercice3.py : programme principal